nsys2prv: translate Nsight System traces to Paraver

Marc Clascà
marc.clasca@bsc.es

Best Practices for Performance and Programmability

Barcelona Supercomputing Center
Centro Nacional de Supercomputación

…but why?

Tools overview

Extrae and Paraver

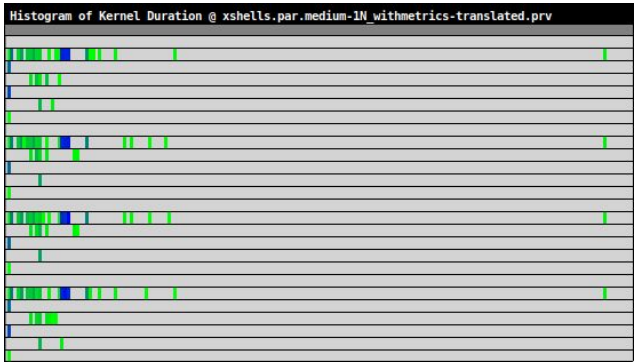
- Parallel performance analysis
- Configurable visualization
- Trace information -> Semantics
- Missing kernel names
- Limited to MPI distributed parallel model - **relevant for LLMs**

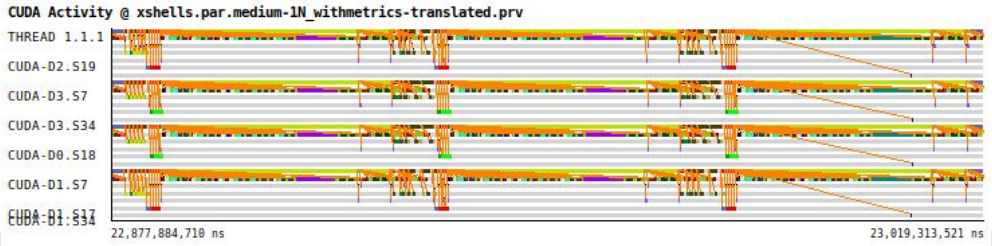
NVIDIA Nsight Systems/Compute

- NVIDIA libraries -> More information (for the moment)
- NOT intended to be a parallel profiler
  - Not good visualization for a lot of processes (imo)
  - Communication semantics is “hidden” in the UI
  - Different report for each process for multi-node
- … but outputs all information in SQLite and CSV format :)

nsys2prv tool: translate nsys traces to Paraver format

- Python script
- Reads data in CSV format from `nsys stats` command and SQLite DB
  - CUDA API calls
  - Kernels and CUDA Graphs, and associated execution information
  - Memory transfers: from/to relationship, size
  - MPI Calls
  - NVTX regions
  - GPU metrics
  - OpenACC
- Custom CFGs, more to come





Benefits and limitations

Benefits:

- Profit from all Nsight information + adapting it to Paraver semantics
- Easy data treatment with python
- Supports analyst work: fast tool feature integration -> first script, then to *Extrae*

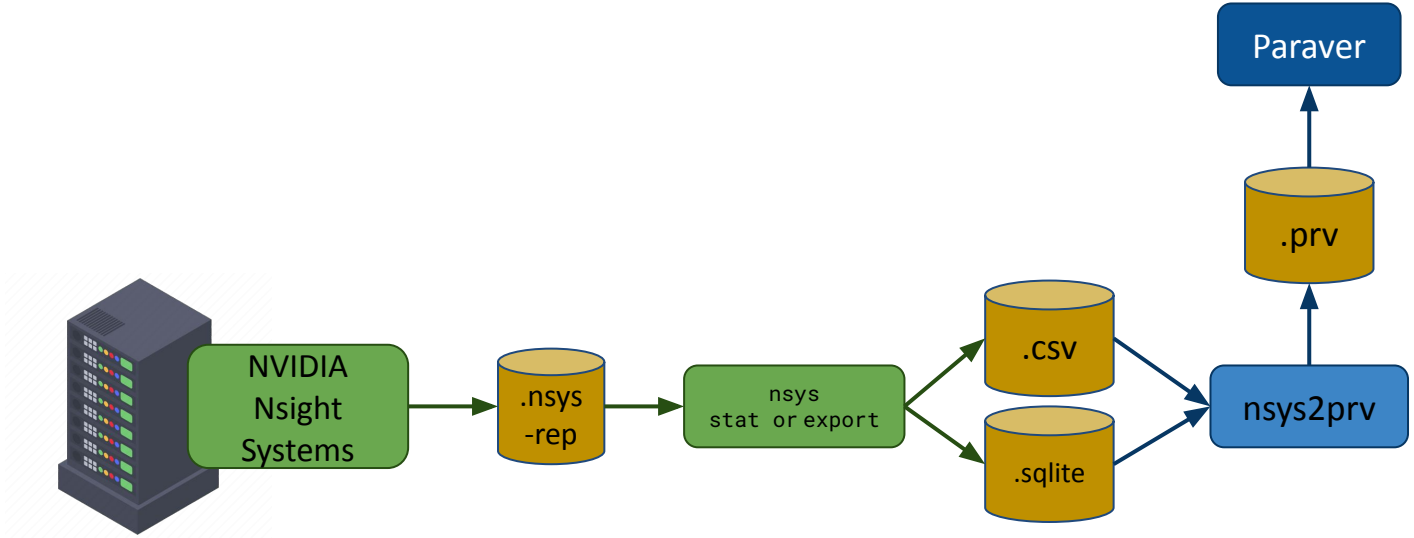
Main enabler for performance analysis: reduce gaps of different tools

Limitations:

- Nsight Systems CSV output format not documented
- SQLite schema briefly documented, and rapidly updated/changed
- Still it's a lot of information, requires manual parsing
- Somehow unstable, lots of edge cases to find yet
- (still) Limited to NVIDIA software stack

Usage

Workflow of the translation



Usage

First, we need to get the Nsight Systems profiling with nsys profile.

```
$> nsys profile --gpu-metrics-devices=cuda-visible -t cuda,nvtx -o ./llm_all
--capture-range=nvtx --env-var=NSYS_NVTX_PROFILER_REGISTER_ONLY=0
--nvtx-capture=RANGE_NAME python TestLLAMA.py
```

In this example, we ask the profiler to only trace during the "RANGE_NAME" NVTX range, to get a trace for our phase of interest.

This should output an .nsys-rep file

```
$> ls
 llm_all.nsys-rep
```

Usage

Now the translator digests the report and outputs a Paraver trace.

- The “-t” flag tells which information do we want to translate.

```
$> nsys2prv -t nvtx_pushpop_trace,cuda_api_trace,gpu_metrics \
    ./llm_all.nsys-rep llm_translated
```

Source report

Output paraver trace

Information to be translated

Usage (multi-report translation)

When running multi-node, we are forced to get multiple reports from Nsight Systems. We can give all reports to *nsys2prv*:

- Enabling analysis of LLMs at scale!

```
$> nsys2prv -t nvtx_pushpop_trace,cuda_api_trace,gpu_metrics \
     -m ./llm_0.nsys-rep ./llm_1.nsys-rep ./llm_2.nsys-rep … llm_translated
```

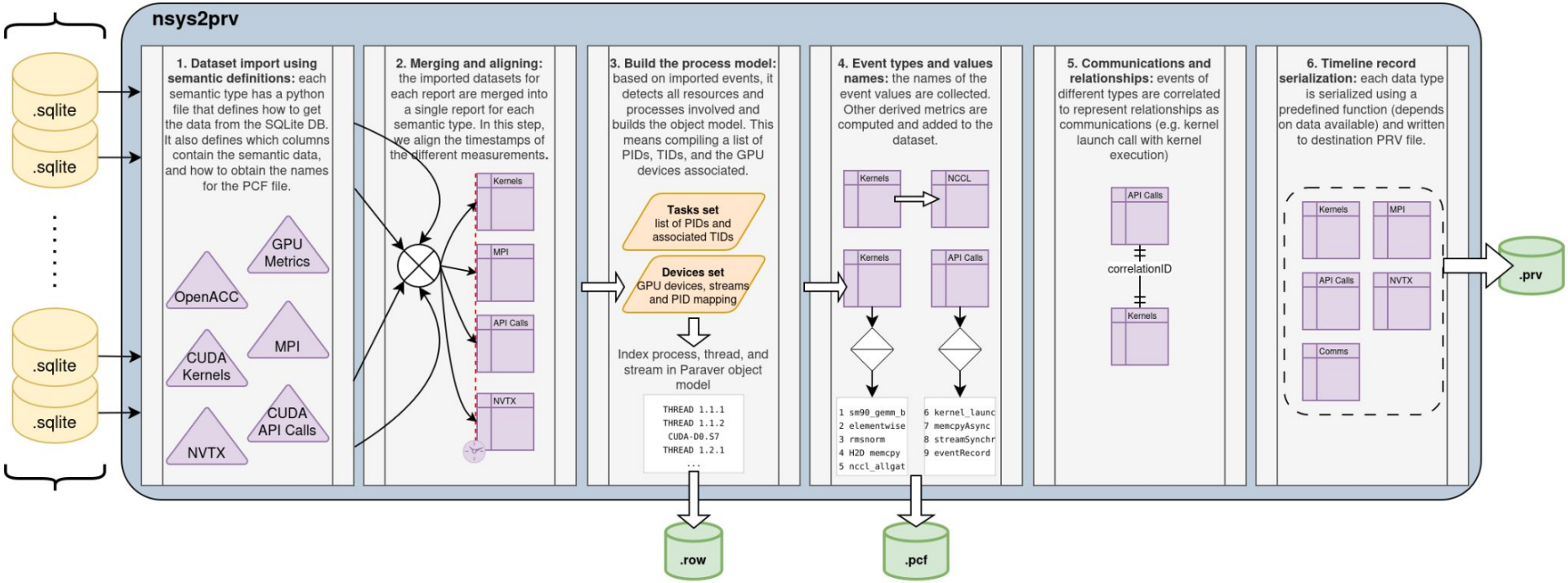
Multi-report flag

Source reports

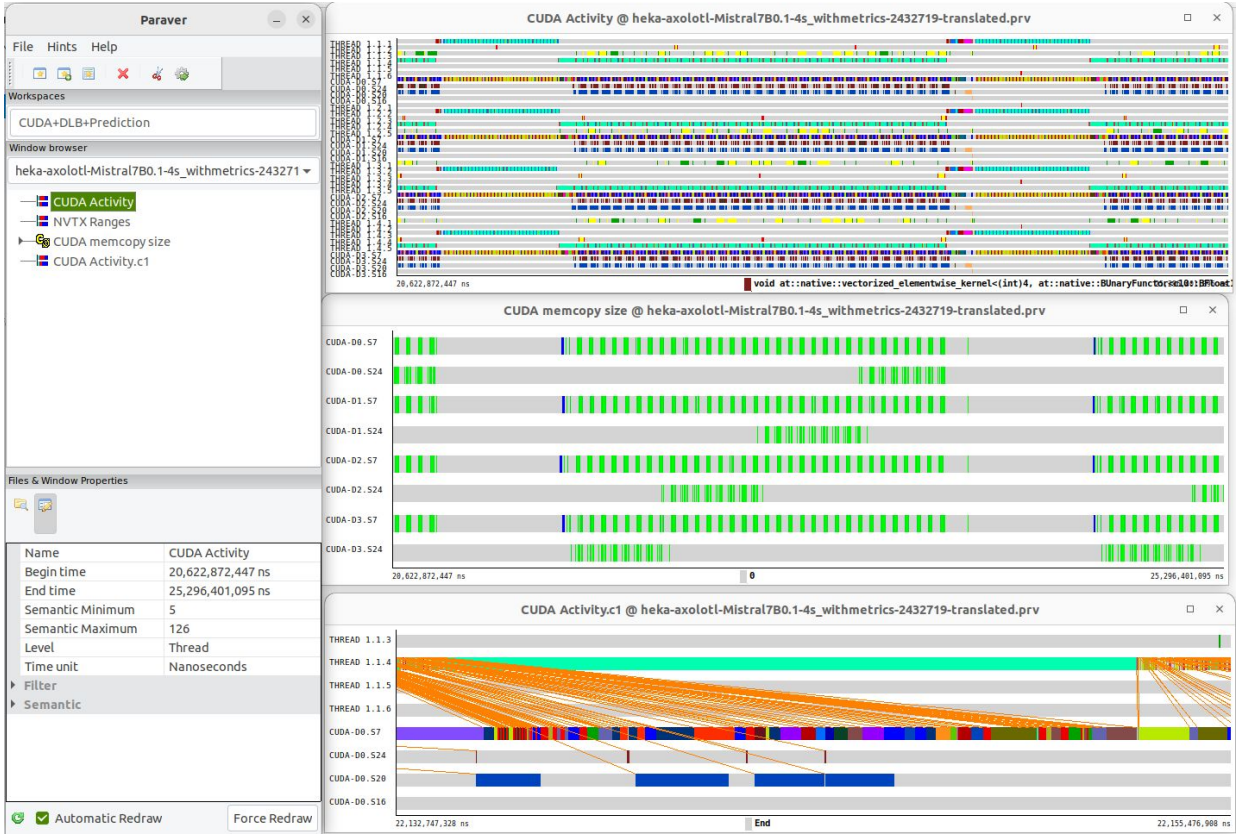
- ⚠️ Inter-node clock synchronization still not implemented

Usage

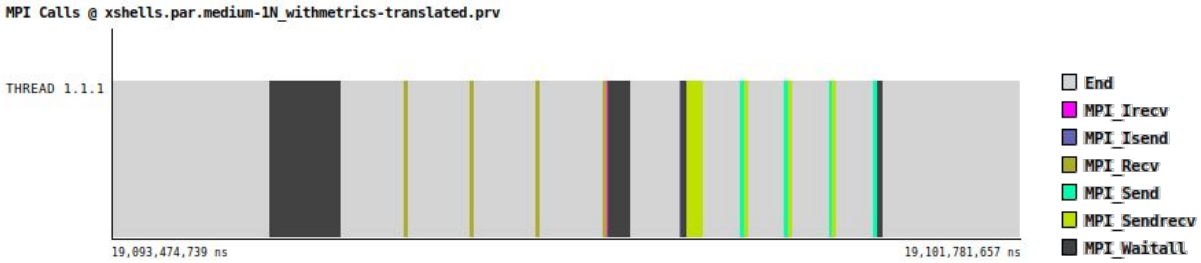
Translation internal process

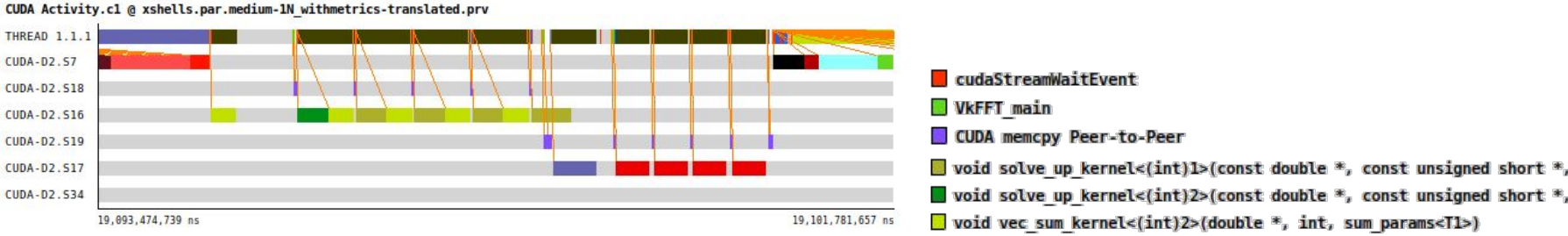


Examples: LLM training

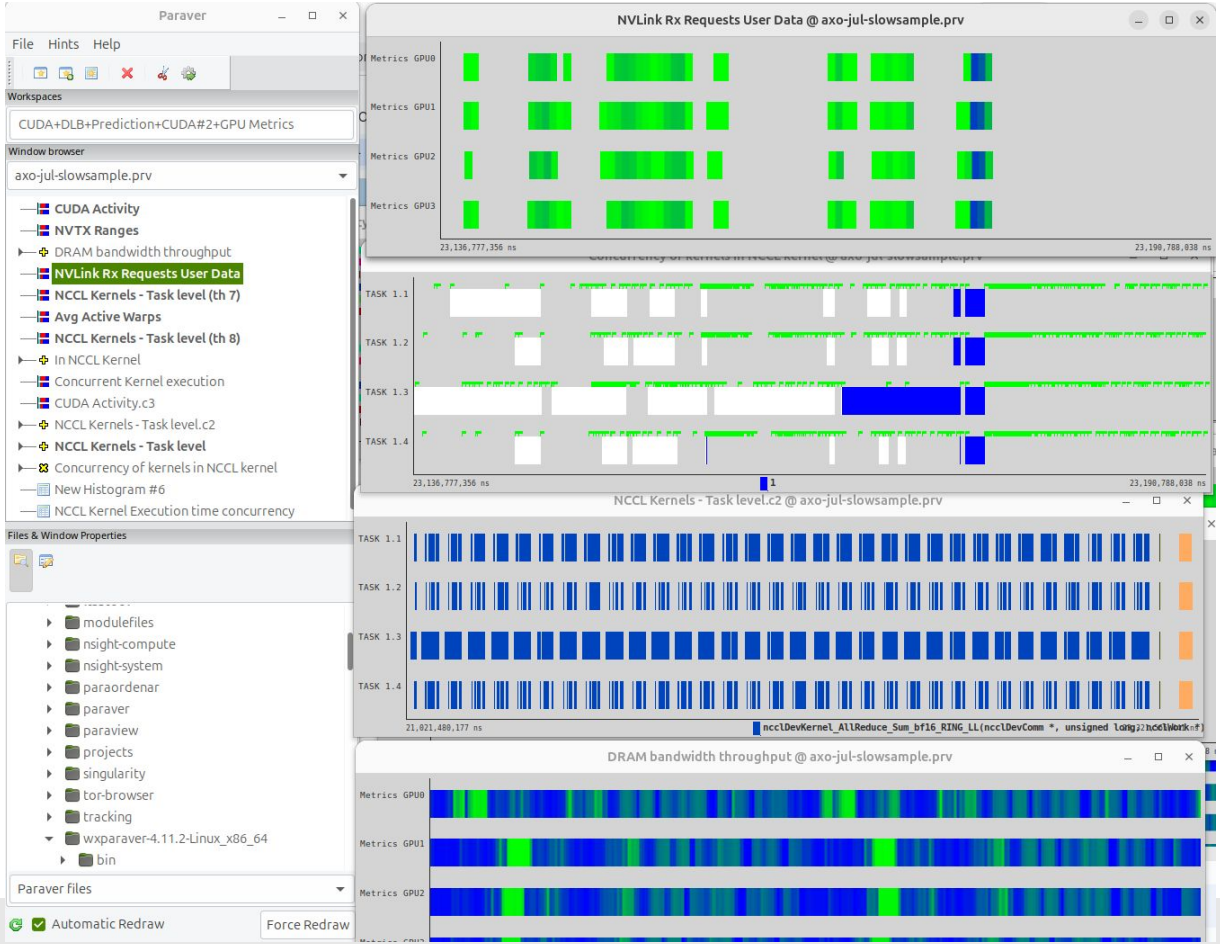


Examples: SOD2D





Examples: NCCL usage study

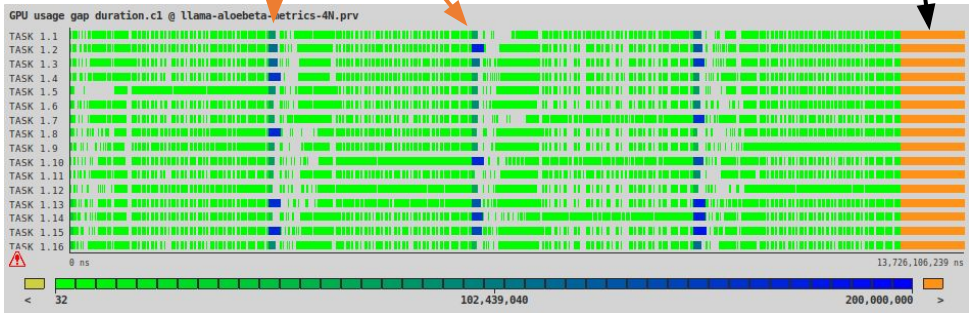


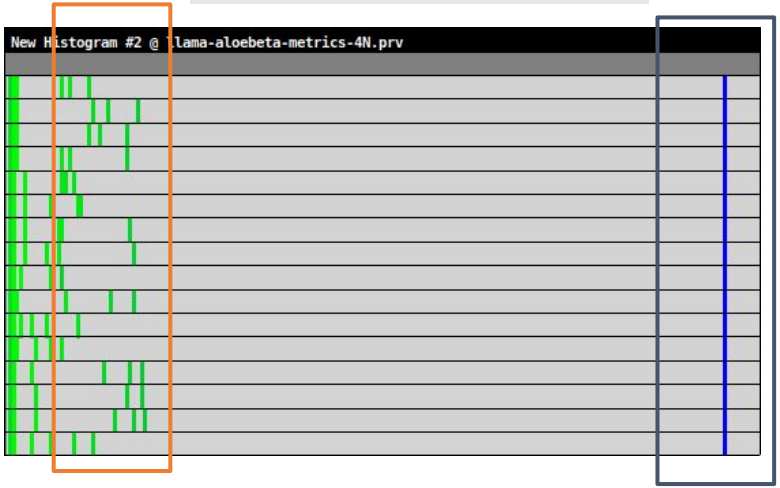
Examples: GPU gap analysis

Significative GPU usage gaps of ~200ms max found at the end of every micro-step, with differences between GPUs

Big GPU gap at the end of step. For further analysis, a complete trace of two steps is needed to have a clean view of what is happening in between 2 steps

Histogram shows relevant GPU gap regions, that then can be related to a timeline window



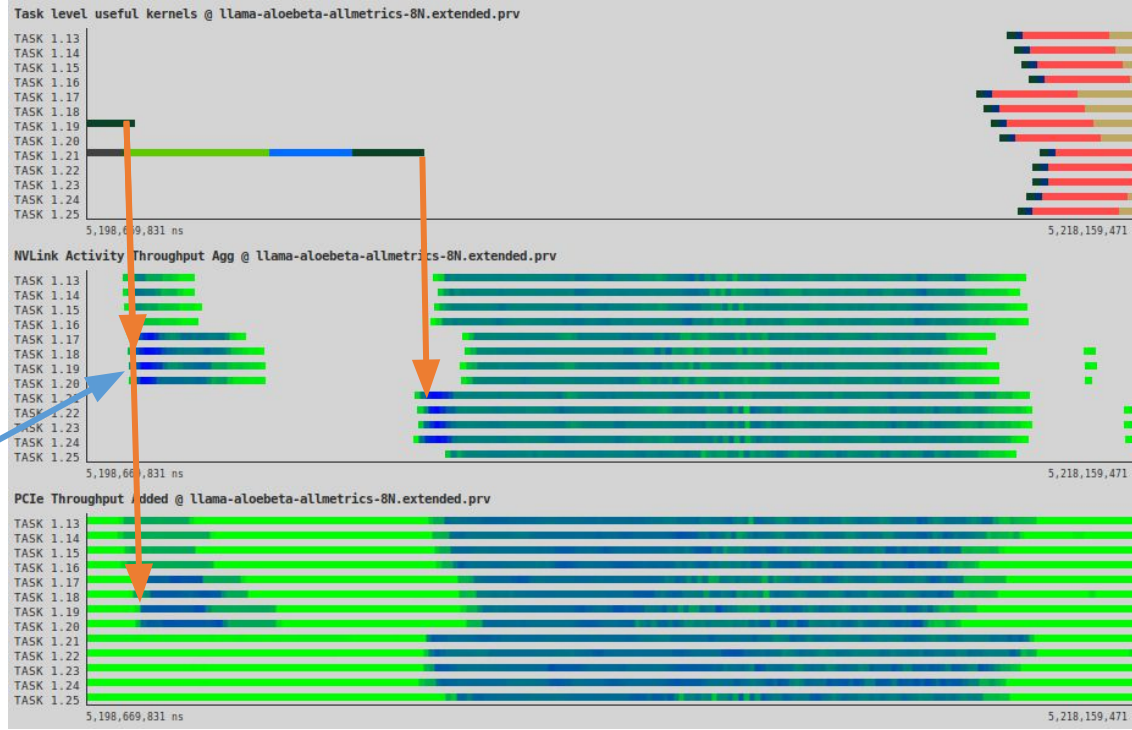


Examples: detailed communication behavior

Compute kernels

NVL Activity throughput aggregated

PCIe throughput aggregated



When a full node reaches the synchronization point, some communication actually happens intra and inter-node, transmitting some data in advance

Let’s get into it!

Common steps:

```
$> module load intel mkl python/3.12.1 sqlite3 #required modules in MN5
$> module load nvidia-hpc-sdk # or ..
$> export /
NSYS_HOME=/gpfs/scratch/nct_310/nsight-systems/opt/nvidia/nsight-systems-cli/2024.6.1
```

🟢 For the stable version:

```
$> source /gpfs/scratch/nct_310/nsys2prv/env/bin/activate
```

🟡 For the prerelease version with latest features:

```
$> source /gpfs/scratch/nct_310/nsys2prv-pre/env/bin/activate
```

Resources

Repo with set of Paraver CFGs:

🔗 https://pm.bsc.es/gitlab/beppp/nsys2prv/-/tree/main/cfgs

Documentation:

🔗 https://pm.bsc.es/gitlab/beppp/nsys2prv/-/wikis/home

Current status of translation features:

🔗 https://pm.bsc.es/gitlab/beppp/nsys2prv/-/wikis/Translation%20status

PIP package:

🔗 https://pypi.org/project/nsys2prv/

Thank you!

Translate a lot of traces and file a lot of bug reports!